Proyecto Final

Lenguajes de Programación II · 2026-2 · Guía del estudiante

Vale	**25 % de la asignatura**
Se hace	En equipos de **3 personas**
Se sustenta	**Jueves 22 de octubre de 2026**, desde las 6:30 pm
Se entrega	**Una sola vez**: el día de la sustentación. No hay entregas parciales

---

1. Qué quiero con este proyecto

Quiero que demuestren **tres cosas**, y todo lo demás de esta guía existe para comprobarlas:

1. **Que saben conectar una página a datos reales.** No datos inventados: los de un servicio que ya existe, con la forma que ya tiene.
2. **Que la página se comporta bien cuando algo sale mal.** Es lo que separa a un estudiante de un profesional: cualquiera hace que funcione cuando todo sale bien; lo difícil es qué ve el usuario cuando el servidor falla, no hay internet o no hay resultados.
3. **Que entienden el código que entregan.** Se puede usar internet, documentación e IA, pero **cada integrante tiene que poder explicar cada línea**. Eso se comprueba en la sustentación.

> **No evalúo cuántas pantallas hicieron ni qué tan bonito quedó.** Una pantalla sencilla que maneja bien los caminos tristes y que ustedes pueden explicar saca más nota que tres pantallas vistosas que solo funcionan cuando todo sale bien.

Para qué sirve esto fuera del curso: es el repositorio que pueden abrir en una entrevista de trabajo, y la habilidad más pedida a un programador de interfaces: consumir una API y manejar sus errores.

---

2. El proyecto, en concreto

Una **aplicación web hecha con HTML, CSS y JavaScript puro** que muestra una **lista de elementos** traída de **una API pública real**, deja **buscar** dentro de ella y maneja bien **los casos en que algo sale mal**.

Un ejemplo, para que se imaginen la pantalla (con la API de libros):

```
Biblioteca                          ← encabezado
[ cien años          ]  [Buscar]    ← búsqueda
42 resultados                       ← cuántos hay

[ portada ]  [ portada ]  [ portada ]   ← una tarjeta por libro,
  Título       Título       Título          con varios datos
  Autor        Autor        Autor
```

Lo que hace el usuario	Lo que debe ver
Abre la página	Un mensaje de «Cargando…» y luego la lista de elementos
Escribe algo y busca	La lista se reemplaza por los resultados, y dice cuántos son
Busca algo que no existe	Un mensaje claro, «No hay resultados», **sin que la página se rompa**
El servidor responde con un error	Un mensaje que dice **qué pasó**, con el código
Se le va el internet	Un mensaje **distinto** que dice que no hay conexión

---

3. Los equipos y sus APIs

Los equipos y las APIs ya están definidos. **No se cambian.**

Equipo	API	Qué tener presente
Equipo 1 (Kevin Rodriguez, David Solarte, Cristian Martinez)	**Open Library** `openlibrary.org`	Libros. Búsqueda: `?q=cien+años`. Datos desordenados: hay libros **sin autor, sin portada y sin año**
Equipo 2 (Camila Gomez, Steven Cortes, Jose Sanchez)	**PokeAPI** `pokeapi.co`	Pokémon. Búsqueda por **nombre exacto**. Sin resultados: **404**. La lista y el detalle son **dos peticiones**
Equipo 3 (Caleb Mora, Michael Laverde, David Bedoya)	**Disney** `api.disneyapi.dev`	Personajes con foto. Búsqueda: `?name=mickey`. Sin resultados: **200** con `data` vacía. Tiene su propia paginación
Equipo 4 (Johan Steven Henao, Harrison Ospina, Cristian Castaño)	**TheMealDB** `themealdb.com`	Recetas. Usa una **clave de prueba «1» escrita en la dirección** (gratis para uso educativo; no hay que registrarse). Sin resultados: **200** con `meals: null`. Los ingredientes vienen como campos sueltos
Equipo 5 (Julian Andres Arbelaez, Santiago Arenas, David Giraldo)	**SWAPI** `swapi.tech`	Personajes de Star Wars. Búsqueda: `?name=sky`. Los datos vienen **envueltos** en `result[].properties`

> **Antes de escribir código, pruebe su API desde el navegador**: que responda, que permita buscar, y **qué devuelve cuando no encuentra nada**. Cada API lo dice de una forma distinta, y de eso depende cómo escriben su manejo de estados.

La API de Rick and Morty no la puede usar ningún equipo: es la de las clases, y el profesor construye con ella el ejemplo resuelto. **Ustedes aplican la misma receta a su propia API.**

---

4. Qué tiene que hacer la aplicación

Cada requisito trae «cómo sé que está bien»: la prueba que haré en la sustentación.

Requisito 1 · Muestra la lista, traída de su API

- Cada elemento se ve en una **tarjeta** con **al menos tres datos** (por ejemplo, nombre, imagen y un dato propio de su API).
- Las tarjetas **no están escritas a mano** en el HTML: se generan con JavaScript a partir de los datos.
- Si un dato **no viene** (un libro sin autor, un personaje sin foto), la tarjeta **no se rompe** y muestra «Sin dato».

Cómo sé que está bien: busco un elemento al que le falte un dato y la tarjeta se ve completa.

Requisito 2 · Se puede buscar

- Hay un campo de texto y la búsqueda **la hace la API** (con su parámetro de búsqueda), no se traen todos los datos para filtrarlos en el navegador.

Cómo sé que está bien: busco algo que sé que existe y aparece; busco algo que no existe y no pasa nada raro.

Requisito 3 · Dice cuántos resultados hay

- Un texto como «42 resultados», que cambia con cada búsqueda.

Cómo sé que está bien: busco dos cosas distintas y el número cambia.

Requisito 4 · Maneja los cinco estados

Es **el requisito que más pesa**, y se explica en la sección 5.

Requisito 5 · El código está organizado

- **Al menos tres archivos de JavaScript**, cada uno con **una responsabilidad**: por ejemplo, uno pide los datos a la API, otro los pinta en pantalla y otro arranca la página.
- **Funciones pequeñas con nombres claros**, y sin código sobrante ni líneas repetidas.
- **El código se escribe en inglés** (nombres de variables y funciones). **Los textos que ve el usuario, en español.**

Cómo sé que está bien: abro un archivo y su nombre dice qué hay adentro. Les pido una función y la encuentran en segundos.

Requisito 6 · Se ve bien en un celular

- Se adapta desde **un celular de 375 px** hasta un computador, **sin barra de desplazamiento horizontal**.
- Las tarjetas se organizan en una **cuadrícula** (`grid`) que cambia de columnas sola.
- **El CSS usa unidades relativas** (`rem`, `em`, `%`, `fr`) y **no píxeles**.

Cómo sé que está bien: achico la ventana a 375 px y aumento el tamaño de letra del navegador, y la página sigue funcionando.

Requisito 7 · Es accesible

- Usa las **etiquetas con significado** (`header`, `main`, `section`, `article`, `nav`, `button`), no solo `div`.
- Todo campo tiene su `label`, toda imagen su `alt`, y los mensajes que cambian llevan `role="status"`.

- Se puede usar **con el teclado** (Tab), y se **ve** dónde está el foco.
- El texto tiene buen contraste.

Cómo sé que está bien: recorro la página con la tecla Tab y llego a todo; veo dónde estoy.

---

5. Los cinco estados

Una página que consulta una API pasa por **cinco situaciones**. **Cada una tiene que verse distinta y decirle algo claro al usuario.**

#	Estado	Qué pasó	Qué debe ver el usuario	Cómo lo provoco para probarlo
1	**Cargando**	La petición está en camino	Un mensaje o un indicador de carga (spinner), y los botones apagados	F12 → Network → «Slow 4G»
2	**Con datos**	Llegaron los datos	La lista, y cuántos resultados hay	Una búsqueda normal
3	**Sin resultados**	La búsqueda no encontró nada	«No hay resultados para…»	Buscar `zzzzqq`
4	**Error del servidor**	El servidor respondió, pero mal (404, 500…)	Un mensaje con **el código** del error	Cambiar la dirección a una que no exista
5	**Sin conexión**	Ni siquiera hubo respuesta	Un mensaje **distinto** del anterior	Apagar el wifi

Dos reglas que se pierden fácil:

- **Se borra lo anterior** antes de pintar lo nuevo, y se muestra que se está trabajando.
- **Después de un error, la página tiene que seguir funcionando**: poder volver a buscar sin recargarla.

El estado 3 depende de la API que les tocó

«Sin resultados» **no llega igual en todas**, y **su código tiene que traducir la forma que les tocó a un mensaje que una persona entienda**. Es la mitad del trabajo de un Front-End.

API	Cómo avisa que no encontró nada
Open Library	**200** con `numFound: 0`
PokeAPI	**404**
Disney	**200** con `data: []`
TheMealDB	**200** con `meals: null`
SWAPI	**200** con `result: []`

> **Aquí aparece una trampa real:** si la API responde 404, `fetch` **no falla**. Hay que revisar `response.ok`. Y si responde 200 con una lista vacía, **no es un error**: es un resultado válido, «cero». No es lo mismo, y su código debe distinguirlos.

---

6. Lo que NO se pide

- **No hay que desplegarla en internet.** La sustentación es en su computador.
- **No se usa React ni librerías de componentes ni Bootstrap.** Es JavaScript puro: lo que se evalúa es **su** código y **su** CSS.
- **No hay inicio de sesión ni backend propio.**
- **No hay que escribir pruebas automatizadas.**
- **No hay que hacer más de una pantalla.** Más pantallas son bienvenidas, **pero no dan más nota**.

> **Si algo de esa lista les tienta, resístanlo.** Una pantalla que maneja bien los cinco estados vale mucho más que tres que solo funcionan cuando todo sale bien.

---

7. Qué se entrega

El jueves 22 de octubre, antes de las 6:30 pm, se entrega **el enlace del repositorio** en Moodle. Es la única entrega.

```
lp2-proyecto-<nombre>/
├── index.html
├── css/style.css
├── js/
│   ├── api.js          ← pide los datos
│   ├── ui.js           ← pinta en pantalla
│   └── main.js         ← arranca la página
└── README.md
```

- **Un solo repositorio por equipo**, llamado `lp2-proyecto-<nombre>`, y **los tres integrantes suben código**.
- Los nombres de los archivos de JavaScript son un ejemplo: lo que se pide es que **haya al menos tres, con una responsabilidad cada uno**.

El `README.md` (máximo veinte líneas)

Tiene que decir **tres cosas**:

1. **Qué hace la aplicación** y **con qué API** trabaja.
2. **Cómo se abre**: con Live Server, desde qué archivo.
3. **Cómo se provoca cada estado** (los de la tabla de la sección 5), para que quien la pruebe pueda verlos.

> **La prueba del `README`:** que otra persona pueda clonar el repositorio y hacerlo funcionar **sin preguntarles nada**. Si no puede, el `README` está incompleto.

Lo que no se sube

- La carpeta `node_modules` (si la hubiera).
- Contraseñas o claves personales.

---

8. Cómo se califica

La nota del proyecto es la de la sustentación, y es la misma para los tres integrantes.

Criterio	Puntos
La aplicación funciona en vivo: consume datos reales y los muestra	**1.0**
Los estados: cargando, sin resultados, error del servidor y sin conexión, cada uno con su mensaje	**1.0**
Interfaz: etiquetas con significado, accesible, responsiva, con contraste y foco visible	**1.0**
Código organizado: archivos con una responsabilidad cada uno, funciones con nombre claro, en inglés	**0.5**
Defensa técnica: explican sus decisiones y responden por su código	**1.0**
Repositorio: `README.md` completo y *commits* de todos	**0.5**
	5.0

> **Dos de los cinco puntos —los estados y la defensa— no se ven en una captura de pantalla.** Una aplicación bonita que solo funciona cuando todo sale bien saca 2.5. Una aplicación sencilla que maneja los caminos tristes y que ustedes pueden explicar saca 4.0.

---

9. La sustentación

Cada equipo **muestra su aplicación funcionando**, en su computador, y la **defiende**. Se hacen **tres cosas**, y **se las decimos desde ahora** para que las preparen:

1. **«Apaguen el internet y muéstrenme qué pasa.»** *(Y se apaga ahí mismo.)* Es el estado 5 en vivo.
2. **«Voy a señalar una línea al azar y me la explican.»** Quien responda puede ser cualquier integrante.
3. **«¿Qué decisión técnica tomaron de la que se arrepienten?»**

> **La defensa técnica se le pregunta a quien menos ha hablado.** Por eso el equipo depende de que los tres entiendan el código: no sirve que uno lo haga todo.

Una prueba que sí harán: cambiar un dato del código (por ejemplo, el texto de un mensaje) y ver si saben dónde tocar.

---

10. Las reglas

- **Se puede usar internet, documentación e IA**, con una condición: **tienen que poder explicar cada línea que entregan.** Se comprueba en la sustentación.
- **Los tres integrantes suben código.** Un repositorio con *commits* de una sola persona cuesta nota en el criterio del repositorio y en el de defensa.
- **El historial de* commits *muestra cuándo y quién trabajó**, aunque no haya entregas parciales: suban su avance a medida que lo hacen, no todo el día anterior.
- **Nadie entrega código que no puede explicar.** Si la explicación no es posible, esa parte no cuenta.

11. Preguntas que siempre salen

¿Podemos usar la API de Rick and Morty?

No: es la de las clases y el profesor la usa como ejemplo.

¿Podemos cambiar de API?

No. Cada equipo trabaja con la que le tocó.

¿Podemos hacer más de una pantalla?

Sí, y no da más nota. Háganlo solo si la principal ya está terminada.

¿Podemos usar Bootstrap o una librería?

No. La rúbrica evalúa **su** CSS y **su** código.

La API que nos tocó cambió o dejó de responder.

Avísenme apenas pase, no el día de la sustentación.

¿Y si no alcanzamos a terminar?

Entreguen lo que haya, **funcionando**. Una pantalla incompleta que maneja bien sus estados saca más que una completa que se cae. Y en la sustentación, **digan ustedes lo que no alcanzaron**: reconocer un límite con precisión se califica dentro de la defensa técnica.

¿La nota es igual para los tres?

La del proyecto sí. La defensa técnica se le pregunta a quien menos ha hablado.